

Overview of Layer normalization

Subject: Layer normalization

1.

Iterate $x \leftarrow \frac{1}{Wx} Wx$ to convergence x^* . This is the eigenvector of W with eigenvalue W^s .

2.

where each network module can be a linear transform, a nonlinear activation function, a convolution, etc. $x^{(0)}$ is the input vector, $x^{(1)}$ is the output vector from the first module, etc. BatchNorm is a module that can be inserted at any point in the feedforward network. For example, suppose it is inserted just after $x^{(l)}$, then the network would operate accordingly:

3.

$b_c^{(l)}$ is the bias term for the c -th channel of the l -th layer. In order to preserve the translational invariance, BatchNorm treats all outputs from the same kernel in the same batch as more data in a batch. That is, it is applied once per kernel c (equivalently, once per channel c), not per activation $x_{h,w,c}^{(l+1)}$:

4.

$\hat{x}_i = x_i - \mu$, $y_i = \gamma \hat{x}_i + \beta$
 $\sigma^2 = \frac{1}{n} \sum (x_i - \mu)^2$, $\sigma = \sqrt{\sigma^2 + \epsilon}$, $y_i = \gamma \hat{x}_i + \beta$

5.

Adaptive instance normalization Adaptive instance normalization (AdaIN) is a variant of instance normalization, designed specifically for neural style transfer with CNNs, rather than just CNNs in general. In the AdaIN method of style transfer, we take a CNN and two input images, one for content and one for style. Each image is processed through the same CNN, and at a certain layer l , AdaIN is applied. Let $x^{(l)}$, content be the activation in the content image, and $x^{(l)}$, style be the activation in the style image. Then, AdaIN first computes the mean and variance of the activations of the content image $x'^{(l)}$, then uses those as the γ , β for InstanceNorm on $x^{(l)}$, content. Note that $x^{(l)}$, style itself remains unchanged. Explicitly, we have:

6.

The BatchNorm module does not operate over individual inputs. Instead, it must operate over one batch of inputs at a time. Concretely, suppose we have a batch of inputs $x^{(1)}(0)$, $x^{(2)}(0)$, ..., $x^{(B)}(0)$, fed all at once into the network. We would obtain in the middle of the network some vectors:

7.

$x_{h,w,c}^{(l)}$ is the activation of the neuron at position (h, w) in the c -th channel of the l -th layer.

8.

$$\mu = \alpha E[x] + (1 - \alpha) \mu_{\text{train}}, \sigma^2 = (\alpha E[x^2] + (1 - \alpha) \mu_{\text{train}}^2 - \mu^2)$$

9.

$$x_{h,w,c}^{(l)} = \sum_{h',w',c'} K_{h'-h,w'-w,c'-c}^{(l)} x_{h',w',c'}^{(l-1)} + b_c^{(l)}$$

10.

Special cases The original paper recommended to only use BatchNorms after a linear transform, not after a nonlinear activation. That is, $\phi(\text{BN}(Wx + b))$, not $\text{BN}(\phi(Wx + b))$. Also, the bias b does not matter, since it would be canceled by the subsequent mean subtraction, so it is of the form $\text{BN}(Wx)$. That is, if a BatchNorm is preceded by a linear transform, then that linear transform's bias term is set to zero. For convolutional neural networks (CNNs), BatchNorm must preserve the translation-invariance of these models, meaning that it must treat all outputs of the same kernel as if they are different data points within a batch. This is sometimes called Spatial BatchNorm, or BatchNorm2D, or per-channel BatchNorm. Concretely, suppose we have a 2-dimensional convolutional layer defined by:

11.

Essentially, it is LayerNorm where we enforce $\mu, \epsilon = 0$. It is also called L2 normalization. It is a special case of L_p normalization, or power normalization: $\hat{x}_i = \frac{x_i}{(\sum_{j=1}^D |x_j|^p)^{1/p}}$, $y_i = \gamma \hat{x}_i + \beta$ where $p > 0$ is a constant.

12.

The ϵ is a small positive constant such as 10^{-9} added to the variance for numerical stability, to avoid division by zero. Finally, it applies a linear transformation:

13.

where B is the batch size, H is the height of the feature map, and W is the width of the feature map. That is, even though there are only B data points in a batch, all BHW outputs from the kernel in this batch are treated equally. Subsequently, normalization and the linear transform is also done per kernel:

14.

where $\bar{a}_{x,y}^j$ is the average activation in a small window centered on location (x,y) and channel i . The hyperparameters k, n, α, β , and the size of the small window, are picked by using a validation set. Similar methods were called divisive normalization, as they divide activations by a number depending on the activations. They were originally inspired by biology, where it was used to explain nonlinear responses of cortical neurons and nonlinear masking in visual perception. Both kinds of local normalization were obviated by batch normalization, which is a more global form of normalization. Response normalization reappeared in ConvNeXT-2 as global response normalization.